# FleetLogix – Guía de Modernización de Bases de Datos – PI#2

En este proyecto integrador se aborda el caso de la compañía FleetLogix de donde se busca la modernización de su base datos con el fin de volverse mas competitiva y eficaz a la hora de hacer negocios.

El flujo de trabajo es el siguiente: bajo una base de datos en PostgreSQL. Éste es un modelo relacional clásico OLTP. Para empezar con la modernización fue poblar estas tablas con datos sintéticos, que es de lo que va el Avance#1

### Avance#1:

Para este avance la consigna esta en poblar tablas con datos sintéticos. Para esto se cuenta con dos cosas: una base de datos PostgreSQL y el código de Python data_generation. Primero es menester hablar de la estructura de la base datos a trabajar.

Relaciones:

La tabla central de operaciones es **trips (viajes)**, la cual actúa como un punto de encuentro (entidad asociativa) para los vehículos, los conductores y las rutas.

- **Vehículos y Viajes (1: N):** vehicles => trips
  - Un vehículo puede realizar múltiples viajes a lo largo del tiempo, pero un viaje específico (trip_id) se asigna a un solo vehículo (vehicle_id).
- **Conductores y Viajes (1: N):** drivers => trips
  - Un conductor puede hacer muchos viajes, pero un viaje en particular es realizado por un solo conductor (driver_id).
- **Rutas y Viajes (1: N):** routes => trips
  - Una ruta predefinida puede ser transitada en múltiples viajes, pero un viaje específico sigue una única ruta (route_id).
- **Viajes y Entregas (1: N):** trips => deliveries
  - Un viaje consolida múltiples entregas (paquetes o pedidos). Cada entrega individual (delivery_id) pertenece a un único viaje (trip_id).
- **Vehículos y Mantenimiento (1: N):** vehicles => maintenance
  - Un vehículo puede tener múltiples registros de mantenimiento en su historia. Cada registro de mantenimiento pertenece a un solo vehículo (vehicle_id).

- Constraints:

**Identidad y Relaciones Estructurales (Primary Keys y Foreign Keys)**
Para empezar, absolutamente todas las tablas cuentan con una clave primaria autoincremental (PRIMARY KEY), lo que garantiza que cada vehículo, conductor,

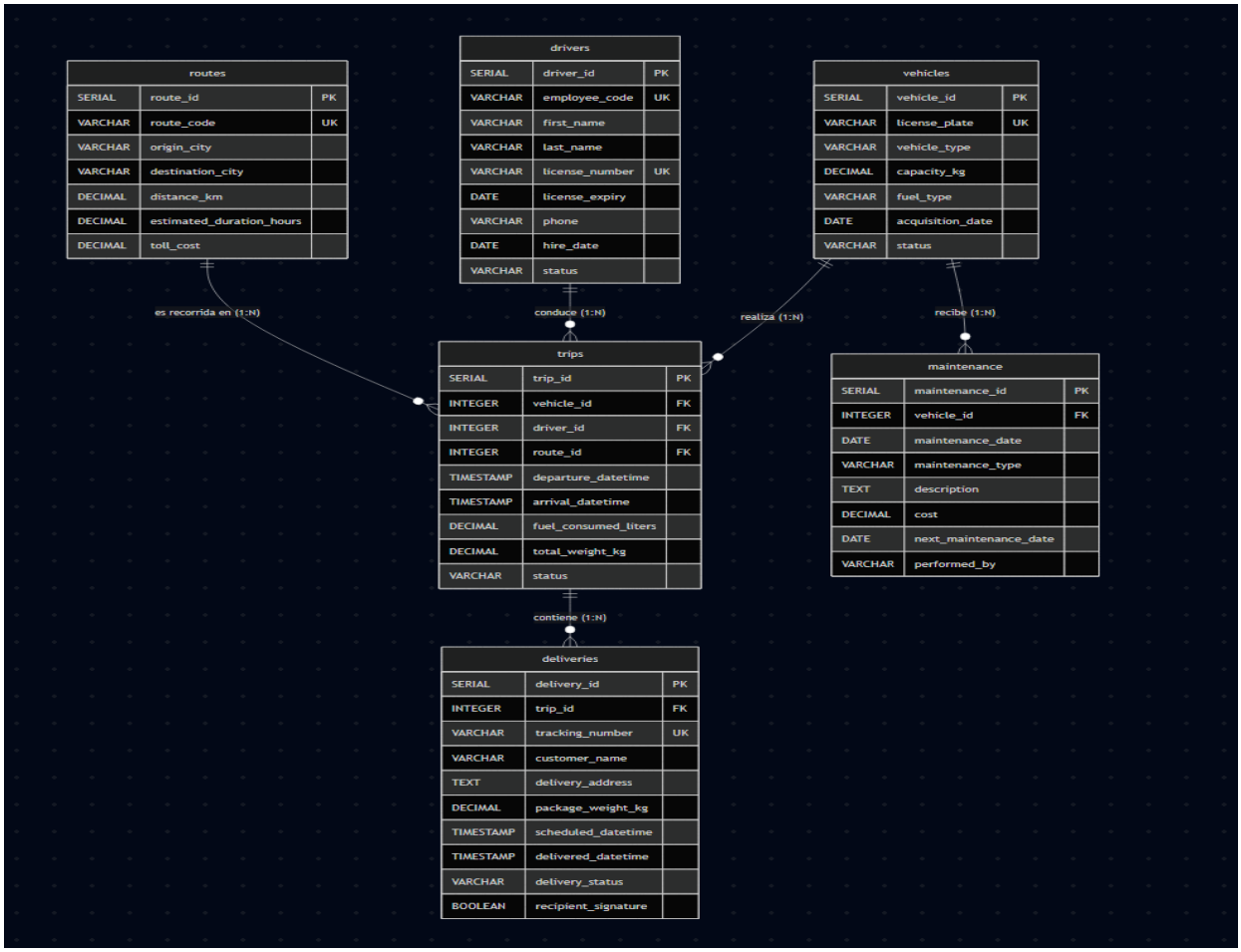
ruta, viaje, entrega y mantenimiento tenga un identificador interno único e irrepetible.

**Reglas de Unicidad Comercial (Unique)** A nivel de negocio, la base de datos es estricta con la información que no debe repetirse. Los *constraints* de unicidad (UNIQUE) garantizan que no puedan existir dos vehículos con la misma placa (license_plate), ni dos conductores compartiendo el mismo código de empleado (employee_code) o número de licencia (license_number).

**Obligatoriedad de los Datos (Not Null)** Para evitar registros a medias, el sistema aplica restricciones de no nulidad (NOT NULL) en los campos críticos de la operación. Por ejemplo, al registrar un conductor, siempre se exigirán sus nombres y su licencia; al crear un vehículo, el tipo de vehículo es mandatorio.

**Automatización de Estados (Default)** Finalmente, el sistema facilita la entrada de datos asumiendo valores por defecto (DEFAULT) en campos de estado o lógicos. Si registras un vehículo o un conductor y no especificas su estado, la base de datos asumirá automáticamente que están activos ('active').

Diagrama ER profesional:



Con esto ya en mente, es momento de habla un poco del script de python.

Con este tenemos que este script se encarga de crear datos ficticios, que lleven cierta logica, y llenen las tablas que hemos documentado. Ademas de la documentacion en el codigo como tal, es pertinente hacer incapie a ciertas funciones como tal

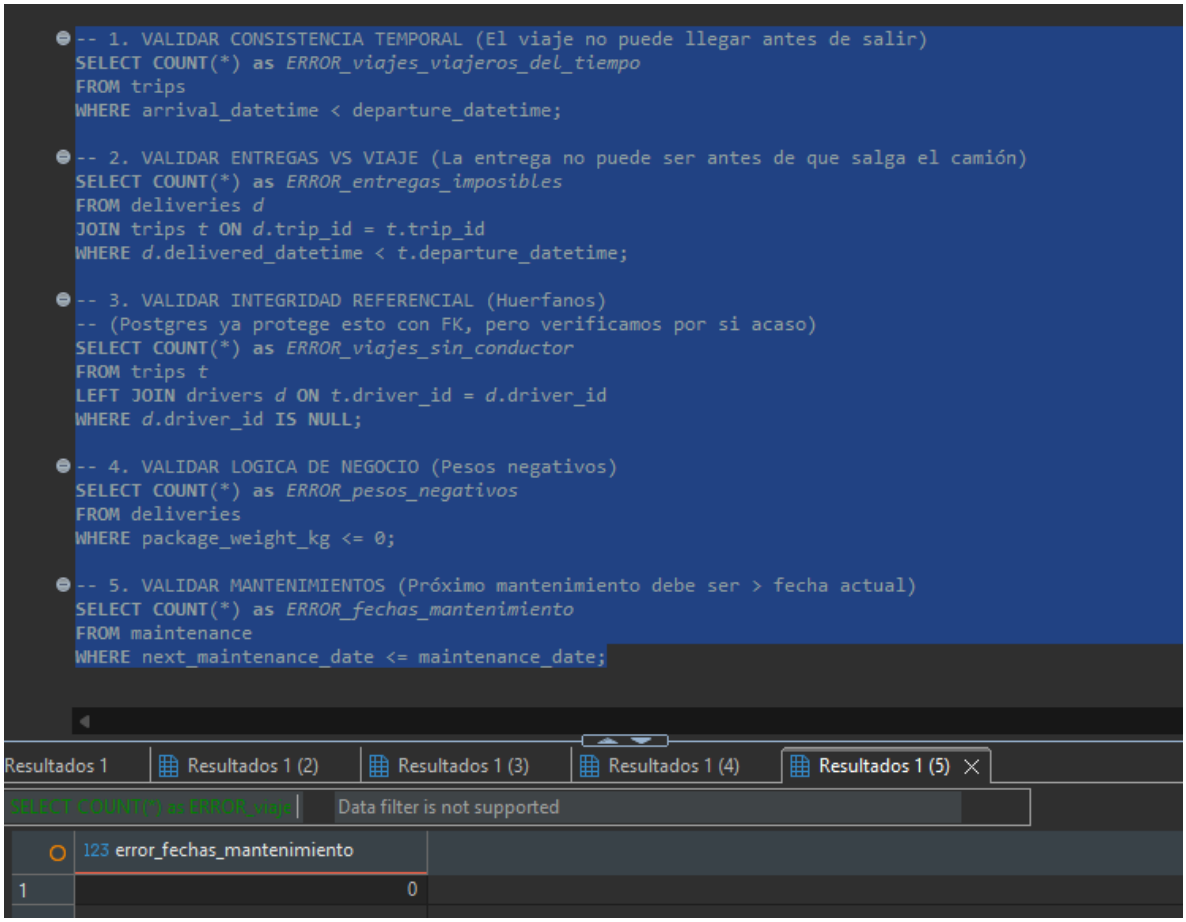
**generate_trips():** Esta función es un simulador que genera un historial de entregas altamente realista a lo largo del tiempo (ej. 100.000 viajes en dos años) consultando primero tus recursos disponibles para luego crear cada trayecto aplicando variables del mundo real, como patrones de horarios, imprevistos en la duración, límites físicos de carga y cálculos de combustible. Todos estos viajes se distribuyen cronológicamente de manera uniforme y se guardan en la base de datos agrupándolos en bloques (*batching*), lo que garantiza un proceso rápido y eficiente sin sobrecargar el servidor con miles de consultas individuales.

**gethourly_distribution():** Esta función construye el "reloj biológico" de la empresa creando un perfil probabilístico de 24 horas que dicta en qué momentos del día se generan más viajes, simulando el comportamiento real del tráfico logístico. Para ello, asigna una probabilidad base baja a las horas de trasnocho o madrugada (2%), aumenta la actividad durante la jornada diurna (de 6 AM a 8 PM), y define dos "horas pico" con el mayor volumen de despachos: mañana (8 AM a 12 PM) y tarde (2 PM a 6 PM); finalmente, normaliza los valores dividiéndolos por su suma total para que representen el 100% (o 1.0), dando al simulador una lista de pesos matemáticamente válida para elegir horarios aleatorios pero realistas.

Otros cambios en el codigo fueron a las funciones ***distribute_weight*** donde se re estructuro para que la suma de las partes diera el total del peso sin pasarse. Ya que la funcion anterior estaba inventando números que no suman el total. Se añadieron validaciones tempranas, metodo de “romper la vara” y controles de precision para hacer que la suma diera el total.

En la funcion ***generate_summary_report,*** se añadieron un seguro contra nulos y un encodign “UTF-8” para evitar errores a la hora de guardar el archivo.

Para asegurar la calidad de los datos se generaron las siguientes querys donde el resultado de todas fue “0”




Ademas de esto, hubieron ciertos erroes en los nombres de las ciudades como MedellÃn, BogotÃi, CamiÃ³n Grande, entre otros, los cuales se solucionaron con un “update” para manterner la calidad.

Avance#2:

Después de haber completado el repositorio de datos OLTP, se harán un total de 8 querys que responderán a problemas de negocio. Analizando su impacto, a qué problemas responden y por último su impacto en cuanto a memoria y procesamiento.

Avance#3:

Para este avance#3, la consigna está en pasar de una base de datos relacional a una base de datos analítica. Esto con el fin de hacer más eficaz la toma de decisiones en base a consultas sobre los datos.

Con esto se apoya bajo un script de Python cuya función es tomar los datos en la base de datos de PostgreSQL hacia la base de datos en la nube de Snowflake.

El primer paso fue otorgarle una consulta maestra para que el script trajera todos los datos crudos listos para transformar. Para el proyecto se usó una muestra de los

últimos 120 días, que son 18125 registros para la carga y transformación en la base de datos OLAP.

Para la transformación de los datos se usó la siguiente lógica para las reglas de negocio en la base de datos OLAP:

### 1. Nivel de Dificultad de la Ruta (difficulty_level)

En logística, la dificultad no solo va dedistancia, sino del tiempo de conducción y el tipo de vía (cobro de peajes).

Podemos usar esta matriz de reglas:

- **Fácil:** Rutas cortas y directas. Distancia < 50 km **y** una duración menor a 1.5 horas.
- **Difícil:** Rutas de larga distancia o complejas. Distancia > a 200 km **o** duración mayor a 4 horas.
- **Medio:** punto intermedio (entre 50 km y 200 km).

### 2. Categoría de Rendimiento del Conductor (performance_category)

En el mundo real, el rendimiento de un conductor se mide cruzando su tasa de entregas a tiempo (OTD - *On-Time Delivery*) con su eficiencia de combustible. Como estamos poblando una dimensión (dim_driver), podemos establecer una regla basada en su experiencia y su historial reciente, o para simplificar el ETL diario, evaluarlo por su eficiencia en el viaje actual:

- **Alto:** Entregó a tiempo (is_on_time = True) **y** su eficiencia de combustible supera la media (por ejemplo, más de 3.5 km por litro).
- **Bajo:** Tuvo retrasos significativos (retraso mayor a 60 minutos) **o** el paquete sufrió daños/problemas (delivery_status = 'failed' o 'damaged').
- **Medio:** Cumplió con la entrega, pero con eficiencia estándar o retrasos menores.

### 3. Entregas a Tiempo y Retrasos (delay_minutes e is_on_time)

La métrica reina aquí es el OTIF (*On-Time In-Full*). Rara vez se exige exactitud al minuto; existe una "ventana de tolerancia".

- **delay_minutes:** Diferencia en minutos entre delivered_datetime y scheduled_datetime. (Si llega antes, el valor puede ser negativo o cero).
- **is_on_time:** En la industria, se suele usar una tolerancia de 15 a 30 minutos. Definiremos que is_on_time es **True** si delay_minutes es menor o igual a 15.

### 4. Métricas de Tiempo y Desgaste (age_months y experience_months)

Estas son dimensiones de cambio lento (SCD) clásicas.

- **age_months (Vehículos):** La diferencia en meses entre la fecha actual de ejecución del ETL y la acquisition_date. Un vehículo con más de 60 meses (5 años) suele cambiar de estatus de mantenimiento.
- **experience_months (Conductores):** La diferencia en meses entre la fecha actual y su hire_date.

### 5. Eficiencia de Combustible (fuel_efficiency_km_per_liter)

Es un cálculo directo crucial para el control de costos (el combustible representa casi el 40% de los gastos operativos de una flota).

- **Fórmula:** distance_km / fuel_consumed_liters.

Avance#4:

Ya para el final se abordará la base de datos a una arquitectura Cloud en AWS. La idea es que reciba y procese datos de la flota en tiempo real. Para esto mismo se van a usar funciones Lambda y se configurará un API Gateway como punto de entrada para las apps móviles de los conductores. La idea de este punto es que el procesamiento de (no toda) la información de la flota tenga un flujo de información en “Streaming” para que los conductores tengas información rápida y precisa para su uso en las calles. Mas de esto en la documentación del avance #4.